UNIVERSIDADE FEDERAL DE SERGIPE

Pilhas: conceito e implementação

Só se põe e se tira por uma das pontas

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje

- Pilha: o último a entrar é o primeiro a sair
- Pilha num vetor: o topo é `v[n-1]`
- Pilha vazia, pilha cheia: os dois erros que não avisam
- Quando o vetor enche: dobrar, não somar um
- Pilha numa lista: o topo é o primeiro nó
- Quanto custa: vetor $\times$ lista
- Exercícios

A pergunta de hoje

O problema: quem fecha quem?

Até aqui

Com `ini` e `fim`, inserir e remover nas pontas da lista ficou barato. E na aula de recursão, a pilha de execução guardava cada chamada até ela terminar — a última a começar era a primeira a acabar.

O que vamos descobrir

Muitos problemas só precisam de **uma** ponta: guardar coisas e depois pegá-las na ordem inversa. Uma estrutura que só faz isso é mais simples — e mais rápida — do que uma lista.

A pergunta de hoje

O compilador recusa `v[f(x)]` se faltar um `]`. Como um programa decide se `( [ { } ] )` fecham na ordem certa — e o que ele precisa guardar enquanto lê?

O programa de hoje: `balanco.c` (1/2)

```c
/* balanco.c --- ( [ { fecham
   na ordem certa? */
#include <stdio.h>

#define MAX 100

typedef struct {
    char v[MAX]; /* topo: v[n-1] */
    int n;       /* quantos ha */
} Pilha;

void empilha(Pilha *p, char c)
{
    p->v[p->n] = c;
    p->n++;
}
```

```c
char desempilha(Pilha *p)
{
    p->n--;
    return p->v[p->n];
}

int vazia(const Pilha *p)
{
    return p->n == 0;
}

int combina(char abre, char fecha)
{
    return (abre == '(' && fecha == ')') ||
           (abre == '[' && fecha == ']') ||
           (abre == '{' && fecha == '}');
}
```

balanco.c (2/2): a verificação e o main

```c
int balanceada(const char *s)
{
    Pilha p = {.n = 0};
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{')
            empilha(&p, c);
        else if (c == ')' || c == ']' || c == '}') {
            if (vazia(&p))
                return 0;  /* fechou sem abrir */
            if (!combina(desempilha(&p), c))
                return 0;  /* fechou o par errado */
        }
    }
    return vazia(&p);      /* sobrou aberto? */
}
```

```c
int main(void)
{
    const char *testes[] = {
        "v[f(x)] = {1, 2}",
        "([)]",
        "(()",
        "())",
        "",
    };
    for (int i = 0; i < 5; i++)
        printf("%-18s %s\n", testes[i],
               balanceada(testes[i])
                   ? "sim" : "nao");
    return 0;
}
```

O que este programa imprime?

Antes de compilar, escreva sua resposta

1. Para cada uma das cinco cadeias, `sim` ou `nao`?
2. Em `"([)]"`, o que está no topo da pilha quando chega o `)`?
3. Em `"(()"` e em `"())"`, qual dos três `return` encerra a função?
4. E a cadeia vazia, que não tem nada para fechar?

Compile, execute e compare

```
$ gcc -Wall -Wextra balanco.c -o balanco
$ ./balanco
v[f(x)] = {1, 2}   sim
([)]               nao
(()                nao
())                nao
                   sim
```

O que o programa guarda

Só os **abertos que ainda não fecharam** — e o único que interessa a cada fechamento é o mais recente.

Três maneiras de errar, três `return`: fechar o par errado (`([)]`), sobrar aberto (`(()`) e fechar sem ter aberto (`())`). A cadeia vazia é balanceada.

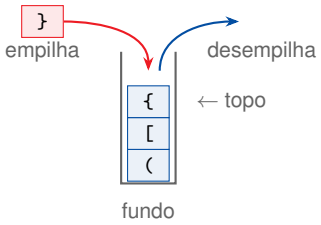
O que é uma pilha

Definição: pilha

Pilha

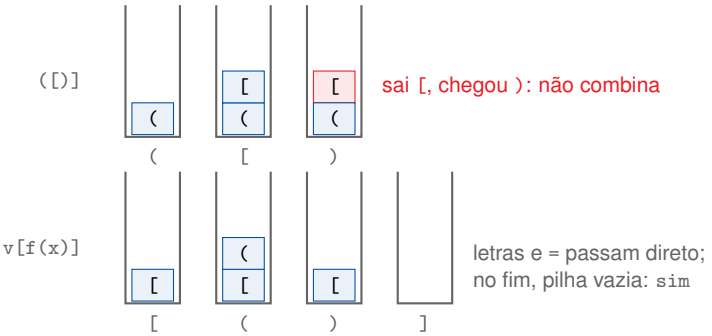
Uma sequência em que inserções e remoções acontecem **só numa das pontas**, o *topo*. O último a entrar é o primeiro a sair: *LIFO*, *last in, first out*.

`empilha(p, x)`	põe `x` no topo
`desempilha(p)`	tira e devolve o topo
`topo(p)`	devolve o topo sem tirar
`vazia(p)`	a pilha não tem nada?



Fonte: Cormen et al. (2009), §10.1 [3]; Celes et al. (2004), cap. 11 [2]

Seguindo a pilha caractere a caractere



Cada fechamento só confere o **topo**: o aberto mais recente ainda sem par. Os de baixo esperam a vez — nenhum precisa ser procurado.

Fonte: Tenenbaum et al. (1995), cap. 2 [10]; Feofiloff (2009), cap. 6 [5]

Pilhas que você já usou

Pilha de execução

Cada chamada empilha um registro de ativação; cada `return` desempilha. Foi assim que `fatorial(4)` voltou na ordem inversa.

Desfazer

Cada edição vai para uma pilha; Ctrl+Z tira a mais recente. Desfazer duas vezes desfaz as duas últimas, na ordem inversa.

Voltar no navegador

Cada página visitada é empilhada; "voltar" desempilha. Abrir um link novo depois de voltar descarta o que estava acima.

Em todos, a pergunta é a mesma: *qual foi a última coisa que ainda não terminou?*

Fonte: Knuth (1997), §2.2.1 [8]; Tenenbaum et al. (1995), cap. 2 [10]

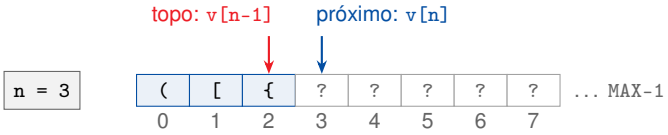
Recapitulando: o que é uma pilha

O que aprendemos

- pilha: inserir e remover só no topo — o último a entrar é o primeiro a sair;
- quatro operações: `empilha`, `desempilha`, `topo` e `vazia`;
- para balancear, basta guardar os abertos ainda sem par: cada fechamento confere só o topo;
- a pilha de execução da recursão é uma pilha no mesmo sentido.

Pilha num vetor

O topo é `v[n-1]`



`empilha`: escreve em `v[n]`, depois `n++`.
`desempilha`: `n--`, depois lê `v[n]`.
`vazia`: `n == 0`.

Nenhuma operação tem laço: cada uma custa um número fixo de passos. O fundo nunca se move — é o topo que anda. Desempilhar não apaga nada: o valor continua lá, fora da pilha.

Fonte: Cormen et al. (2009), §10.1, PUSH e POP [3]; Celes et al. (2004), cap. 11 [2]

E se não conferir a pilha vazia?

```
/* balanco_errada.c: o laco de balanceada */
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{')
            empilha(&p, c);
        else if (c == ')' || c == ']' || c ==
      '}') {
            /* sumiu: if (vazia(&p)) return
     0; */
            if (!combina(desempilha(&p), c))
                return 0;
        }
    }
```

Pense antes de passar o slide

O `main` é o mesmo.

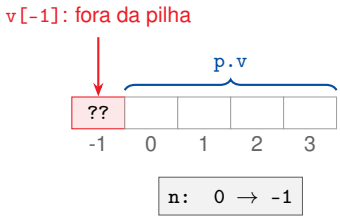
1. Qual das cinco cadeias chega a desempilhar a pilha vazia?
2. Quanto vale `p->n` depois, e que posição de `v` é lida?
3. A resposta impressa muda?

Execute e compare

```
$ ./balanco_errada
v[f(x)] = {1, 2}   sim
([)]               nao
(()                nao
())                nao
                   sim
$ gcc -g -fsanitize=address balanco_errada.c -o balanco_errada-asan
$ ./balanco_errada-asan
==95953==ERROR: AddressSanitizer: stack-buffer-underflow on address 0x00016f39245f
READ of size 1 at 0x00016f39245f thread T0
    #0 0x000100a6cab4 in desempilha balanco_errada.c:20
    #1 0x000100a6cea0 in balanceada balanco_errada.c:44
  This frame has 1 object(s):
    [32, 136) 'p' (line 38) <== Memory access at offset 31 underflows this variable
```

A linha 20 é o `return p->v[p->n];` de `desempilha`. Saída do relatório resumida.

A resposta: certo por sorte é errado



Em `"())"`, o segundo `)` encontra a pilha vazia: `n` vira -1 e `desempilha` lê `v[-1]`, um byte **antes** do vetor.

O byte lido era lixo, `combina` devolveu 0 e a resposta saiu `nao` — a certa, por acaso. Ler fora do vetor é comportamento indefinido: outro compilador, outra resposta.

Desempilhar a pilha vazia tem nome: *underflow*. Quem chama precisa perguntar `vazia` antes.

Fonte: Cormen et al. (2009), §10.1, *underflow* [3]; ISO/IEC 9899:2018, §6.5.6 [6]; Serebryany et al. (2012) [9]

E se a pilha encher?

```c
/* transborda.c: k vezes '(', depois ')' */
int niveis[] = {99, 100, 101, 150};
for (int i = 0; i < 4; i++) {
    Pilha p = {.n = 0};
    int k = niveis[i], fechou = 0;
    for (int j = 0; j < k; j++)
        empilha(&p, '(');
    printf("%3d abertos: n = %3d", k, p.n);
    while (!vazia(&p) && fechou < k) {
        desempilha(&p);
        fechou++;
    }
    int ok = fechou == k && vazia(&p);
    printf(", fechou %3d -> %s\n", fechou,
           ok ? "sim" : "nao");
}
```

Pense antes de passar o slide

`MAX` continua 100.

1. Para `k = 101`, onde o último `(` é escrito?
2. Que valor de `n` você espera ver impresso?
3. O `-fsanitize=address` vai reclamar?

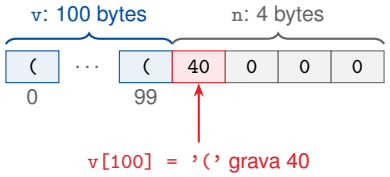
Execute e compare

```
$ ./transborda
 99 abertos: n =  99, fechou  99 -> sim
100 abertos: n = 100, fechou 100 -> sim
101 abertos: n =  41, fechou  41 -> nao
150 abertos: n =  90, fechou  90 -> nao
$ gcc -g -fsanitize=address transborda.c -o transborda-asan && ./transborda-asan
 99 abertos: n =  99, fechou  99 -> sim
100 abertos: n = 100, fechou 100 -> sim
101 abertos: n =  41, fechou  41 -> nao
150 abertos: n =  90, fechou  90 -> nao
```

Empilhou 101, mas `n` diz 41. E nem o *sanitizer* viu nada.

Para 150, `n` termina em 90: os 49 `(` depois do estrago foram escritos *dentro* do vetor, por cima dos antigos.

A resposta: `v[100]` é o primeiro byte de `n`



No registro, `n` vem logo depois dos 100 bytes de `v`. O 101º `empilha` escreve `'('` — código 40 — no byte menos significativo de `n`, que valia 100 (x86-64 e ARM guardam esse byte primeiro); o `n++` seguinte faz 41. A escrita caiu dentro do objeto `p`, e o *sanitizer* não a acusa. Encher a pilha é o *overflow*: quem empilha precisa saber se ainda cabe.

Fonte: ISO/IEC 9899:2018, §6.7.2.1 e §6.5.6 [6]; Cormen et al. (2009), §10.1 [3]; medido com `transborda.c`

Recapitulando: pilha num vetor

O que aprendemos

- o fundo é `v[0]`, o topo é `v[n-1]`; empilhar e desempilhar só mexem em `n` e numa posição;
- desempilhar a vazia (*underflow*) lê fora do vetor; empilhar na cheia (*overflow*) escreve fora;
- os dois podem dar a resposta certa, uma errada ou nenhum aviso — nem do ASan;
- conferir `vazia` antes de tirar e “ainda cabe?” antes de pôr não é opcional.

Quando o vetor enche

Um vetor que cresce: `pilha_vetor.c`

```c
struct pilha {
    int *v;     /* topo: v[n-1] */
    int n;      /* quantos ha */
    int cap;    /* quantos cabem */
};

void empilha(Pilha *p, int x)
{
    if (p->n == p->cap) { /* cheio */
        p->cap *= 2;
        p->v = aloca(p->v,
                     p->cap * sizeof *p->v);
    }
    p->v[p->n] = x;
    p->n++;
}
```

`aloca` é um `realloc` que encerra o programa se faltar memória; `pilha_cria` começa com `cap = 4`.

Pense antes de passar o slide

1. Por que não `p->cap += 1`? Gasta menos memória.
2. Para empilhar n valores a partir de `cap = 1`, quantos elementos são copiados em cada estratégia, se cada `realloc` mudar o bloco de lugar?

Fonte: ISO/IEC 9899:2018, §7.22.3.5 [6]

Execute e compare: somar um $\times$ dobrar

	`cap + 1`		`cap * 2`	
n	cópias	tempo	cópias	tempo
20 000	$2{,}0 \cdot 10^8$	0,05 s	32 767	$<$0,001 s
40 000	$8{,}0 \cdot 10^8$	0,34 s	65 535	$<$0,001 s
80 000	$3{,}2 \cdot 10^9$	0,61 s	131 071	$<$0,001 s
160 000	$1{,}3 \cdot 10^{10}$	1,65 s	262 143	$<$0,001 s

`crescimento.c`, `-O2`; cada crescimento aloca, copia e libera

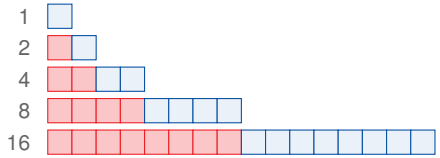
Somando um, dobrar n **quadruplica** as cópias.

Dobrando, as cópias ficam sempre abaixo de $2n$: dobrar n dobra as cópias.

O `realloc` de verdade às vezes aumenta o bloco no lugar e não copia nada — mas a norma não promete isso.

Fonte: medido com `crescimento.c`; ISO/IEC 9899:2018, §7.22.3.5 [6]

A resposta: dobrar paga as cópias adiantado



vermelho: copiado ao crescer

Somando um:
$1 + 2 + \cdots + (n-1) = n(n-1)/2$ cópias.

Dobrando: $1 + 2 + 4 + \cdots + n/2 < n$ cópias.

Cada crescimento copia metade do novo vetor, mas depois dele cabem outros tantos `empilha` sem cópia nenhuma. Dividido entre todos, o custo por `empilha` é constante: o *custo amortizado*.

O preço: até metade do vetor pode ficar vazia.

Fonte: Cormen et al. (2009), §17.4, *dynamic tables* [3]

Recapitulando: quando o vetor enche

O que aprendemos

- guardar `cap` ao lado de `n` e crescer com `realloc` quando `n == cap` acaba com o *overflow*;
- crescer de um em um custa $n^2/2$ cópias; dobrar, menos de n;
- dobrando, um `empilha` raro é caro, mas a média por operação é constante;
- a troca: até metade da memória reservada pode ficar sem uso.

Pilha numa lista

O topo é o primeiro nó: pilha_lista.c

```c
struct pilha {
    No *topo;   /* 1o no, ou NULL */
};

void empilha(Pilha *p, int x)
{   /* insere no inicio */
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) exit(1);
    novo->valor = x;
    novo->prox = p->topo;
    p->topo = novo;
}
```

```c
int desempilha(Pilha *p)
{   /* remove o primeiro */
    if (p->topo == NULL) {
        fprintf(stderr, "pilha vazia\n");
        exit(1);
    }
    No *velho = p->topo;
    int x = velho->valor;
    p->topo = velho->prox;
    free(velho);
    return x;
}
```



o (é o fundo: foi o primeiro a entrar

Fonte: Celes et al. (2004), cap. 11 [2]; Feofiloff (2009), cap. 6 [5]

Por que a lista simples basta

Inserir e remover no **início** eram as duas operações baratas da lista simples: nenhuma precisa de `ant`, de `fim` ou de laço.

Se o topo fosse o *último* nó, empilhar seria barato com `fim`, mas desempilhar exigiria achar o penúltimo — *n* passos, ou um `ant` em cada nó.

Sem *overflow*, sem `cap`

Cada `empilha` pede um nó ao `malloc`: a pilha só enche quando acaba a memória do processo. O *underflow* continua possível — por isso o teste em `desempilha`.

Fonte: Celes et al. (2004), cap. 11 [2]; Cormen et al. (2009), §10.2, exercício 10.2-2 [3]

Uma interface, duas implementações

```c
/* pilha.h */
#ifndef PILHA_H
#define PILHA_H

typedef struct pilha Pilha; /* campos no .c */

Pilha *pilha_cria(void);
void pilha_libera(Pilha *p);

void empilha(Pilha *p, int x);
int desempilha(Pilha *p);
int topo(const Pilha *p);
int vazia(const Pilha *p);

#endif
```

`balanco_tad.c` tem a mesma `balanceada`, agora com `Pilha *`, e testa também 10^5 abertos seguidos de 10^5 fechados.

Pense antes de passar o slide

1. O `main` pode escrever `p->n`? E `sizeof(Pilha)`?
2. O que muda em `balanco_tad.c` para trocar o vetor pela lista?
3. As duas versões imprimem `sim` para 10^5 níveis?

Fonte: Kernighan e Ritchie (1988), §4.5 [7]; Celes et al. (2004), cap. 9 [2]

Execute e compare

```
$ gcc -Wall -Wextra balanco_tad.c pilha_vetor.c -o balanco_vetor
$ gcc -Wall -Wextra balanco_tad.c pilha_lista.c -o balanco_lista
$ ./balanco_vetor                    $ ./balanco_lista
v[f(x)] = {1, 2}   sim               v[f(x)] = {1, 2}   sim
([)]               nao               ([)]               nao
(()                nao               (()                nao
())                nao               ())                nao
                   sim                                  sim
100000 niveis      sim               100000 niveis      sim
```

`balanco_tad.c` não mudou **nenhuma** linha: só o arquivo passado ao `gcc`. E 10^5 níveis já não transbordam.

O `main` só vê um `struct pilha` incompleto: pode guardar um `Pilha *`, mas `p->n` e `sizeof(Pilha)` não compilam.

Recapitulando: pilha numa lista

O que aprendemos

- o topo é o primeiro nó: empilhar é inserir no início, desempilhar é remover o primeiro;
- uma lista simples basta — só uma ponta é usada;
- a lista não enche, mas desempilhar a vazia continua sendo erro;
- com `pilha.h`, quem usa a pilha não sabe nem precisa saber se ela é vetor ou lista.

Quanto custa

Empilhar e desempilhar n valores: vetor $\times$ lista

n	vetor (dobra)	lista
1 000 000	0,002 s	0,035 s
2 000 000	0,003 s	0,123 s
4 000 000	0,005 s	0,127 s
8 000 000	0,009 s	0,234 s

`vetor_vs_lista.c`, `-O2`, valores `int`

As duas crescem em linha reta: cada operação custa um número fixo de passos. Mas a lista é umas **25 vezes** mais lenta.

Na lista, cada `empilha` é um `malloc` e cada `desempilha`, um `free`; os nós ficam espalhados pela memória. No vetor, os valores estão lado a lado e o `realloc` é raro.

Fonte: medido com `vetor_vs_lista.c`; Bryant e O'Hallaron (2016), §6.2 e §9.9 [1]

Vetor ou lista?

	vetor fixo	vetor que dobra	lista encadeada
`empilha`	constante	constante (amortizado)	constante + `malloc`
`desempilha`, `topo`, `vazia`	constante	constante	constante
pode encher?	sim, em `MAX`	só sem memória	só sem memória
memória por `int`	`MAX` $\times$ 4 B	4 a 8 B	16 B + o do `malloc`

Para uma pilha, o vetor que dobra costuma ganhar. A lista compensa quando cada elemento é grande e já mora num nó — ou quando um `empilha` nunca pode demorar mais que os outros.

Fonte: Cormen et al. (2009), §10.1 e §17.4 [3]; Celes et al. (2004), cap. 11 [2]

Recapitulando: quanto custa

O que aprendemos

- nas três versões, toda operação de pilha tem custo fixo — não depende de n;
- o vetor que dobra é o mais rápido na prática: memória contígua e poucas chamadas ao alocador;
- a lista gasta 16 bytes por `int` e um `malloc` por `empilha`;
- o vetor fixo só serve quando há um limite conhecido — e conferido.

Mãos à obra

Altere o programa

Tarefa 1

Faça `balanceada` devolver a **posição** do primeiro erro (ou -1 se estiver tudo certo) e imprima-a. Em `"(()"`, qual posição faz sentido informar?

Tarefa 2

Guarde na pilha a posição de cada aberto (um `int`), em vez do caractere. O que `combina` passa a receber? Que mensagem fica possível em `"(()"`?

Tarefa 3

Troque `MAX` por 4 e acrescente a `empilha` um teste que encerra com mensagem se a pilha estiver cheia. Rode `transborda.c` de novo e preveja cada linha.

Exercícios (1/3)

Exercício 1 — uma pilha que avisa

`int empilha(Pilha *p, char c)` devolve 0 se a pilha está cheia; `int desempilha(Pilha *p, char *c)` devolve 0 se está vazia e o topo por `*c`. Use-as em `balanceada`.

Exercício 2 — inverter uma cadeia

`void inverte(char *s)` com uma pilha de `char`; depois `inverte_palavras`: `"ola mundo"` $\rightarrow$ `"alo odnum"`.

Exercício 3 — decimal para binário

Imprima os dígitos de `n` na base `b` ($2 \leq b \leq 16$): os restos de `n % b` saem do menos significativo para o mais — a pilha devolve na ordem certa.

Fonte: Celes et al. (2004), cap. 11 [2]; Tenenbaum et al. (1995), cap. 2 [10]

Exercícios (2/3)

Exercício 4 — calculadora pós-fixa

Avalie `"3 4 + 2 *"` $\rightarrow$ 14 (pós-fixa, `+ - * /`). Número: empilha. Operador: desempilha dois, empilha o resultado. Qual dos dois é o operando da esquerda?

Exercício 5 — pilha com mínimo

Acrescente `int minimo(const Pilha *p)`, que devolve o menor elemento em tempo constante. Dica: uma segunda pilha de mínimos.

Exercício 6 — duas pilhas num vetor

Num único `int v[MAX]`, uma pilha cresce de `v[0]` para a direita e outra de `v[MAX-1]` para a esquerda. Quando enchem?

Fonte: Feofiloff (2009), cap. 6 [5]; Knuth (1997), §2.2.2 [8]

Exercícios (3/3)

Exercício 7 — desfazer e refazer

Um contador recebe comandos `+k`, `-k`, `u` (desfaz) e `r` (refaz). Use duas pilhas. O que acontece com a pilha de refazer quando chega um `+k` novo?

Exercício 8 — ordenar com uma pilha auxiliar

Ordene uma pilha (menor no topo) usando só uma segunda pilha e uma variável `int`. Quantos `empilha` isso faz no pior caso?

Exercício 9 — a lista de trás para a frente, sem recursão

Imprima uma lista simples do último ao primeiro com uma pilha de `No *`. Compare com a versão recursiva: quem faz o papel da pilha lá?

Fonte: Tenenbaum et al. (1995), cap. 2 [10]; Cormen et al. (2009), §10.1, exercícios [3]

Desafio

Uma calculadora de verdade

Leia expressões *infixas* com inteiros, `+ - * /` e parênteses, uma por linha, e imprima o valor:

- primeiro converta para pós-fixa com uma pilha de **operadores**: um operador desempilha os de precedência maior ou igual antes de entrar; `(` entra sempre; `)` desempilha até o `(`;
- depois avalie com uma pilha de **valores** (Exercício 4);
- parênteses desbalanceados e divisão por zero viram mensagem de erro, nunca *segmentation fault*.

`2 * (3 + 4) - 5` → `2 3 4 + * 5 -` → `9` `(1 + 2` → `erro: parenteses`

Fonte: Dijkstra (1961) [4]; Tenenbaum et al. (1995), cap. 2 [10]

Fechamento

O que mudou hoje

Restringir a estrutura a uma ponta deixou tudo mais simples: quatro operações, nenhum laço, e duas implementações trocáveis atrás de um `.h`.

O que ficou

A pilha devolve na ordem *inversa*. Para atender na ordem de chegada — o primeiro a entrar, o primeiro a sair — é preciso outra estrutura.

Para levar

- pilha: só o topo entra e sai — o último a entrar sai primeiro;
- num vetor: topo em `v[n-1]`; confira vazia antes de tirar e cheia antes de pôr;
- vetor que cresce: dobre, não some um;
- numa lista: topo é o primeiro nó; a lista simples basta.

Próxima aula: filas — põe-se por uma ponta e tira-se pela outra.

Referências I


Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2016.
Seções 6.2 (localidade) e 9.9 (alocação dinâmica).


Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
Introdução a estruturas de dados: com técnicas de programação em C.
Elsevier, Rio de Janeiro, 2004.
Capítulos 9 (tipos abstratos de dados) e 11 (pilhas).

Referências II

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
  Introduction to Algorithms.
  MIT Press, Cambridge, MA, 3 edition, 2009.
  Seções 10.1 (pilhas e filas), 10.2 (listas encadeadas) e 17.4 (tabelas dinâmicas).

- Edsger W. Dijkstra.
  ALGOL 60 translation: an ALGOL 60 translator for the X1 and making a translator for ALGOL 60.
  Technical Report MR 35/61, Mathematisch Centrum, Amsterdam, 1961.
  Origem do algoritmo de conversão infixa $\rightarrow$ pós-fixa com pilha de operadores.

Referências III


Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 6: pilhas (parênteses balanceados e notação posfixa).


ISO/IEC.
ISO/IEC 9899:2018 — Information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seções 6.5.6 (aritmética de ponteiros), 6.7.2.1 (estruturas) e 7.22.3.5 (`realloc`).

Referências IV


Brian W. Kernighan and Dennis M. Ritchie.
The C Programming Language.
Prentice Hall, Englewood Cliffs, NJ, 2 edition, 1988.
Seção 4.5: arquivos de cabeçalho.


Donald E. Knuth.
The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, Reading, MA, 3 edition, 1997.
Seções 2.2.1 (pilhas, filas e deques) e 2.2.2 (alocação sequencial).

Referências V

- Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
  AddressSanitizer: A fast address sanity checker.
  In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
- Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
  Estruturas de dados usando C.
  Pearson Makron Books, São Paulo, 1995.
  Capítulo 2: a pilha.